ADR-011 - BusinessPartner / droppedPartnerIds Resolution via ACL and VMM

Field	Value
ADR ID	ADR-011
Jira	SPARK-30228
Status	DECIDED
Decision Date	09 Jun 2026
Decision Makers	Ashley.Sparks Giri.AlkondanSubbiah Saritha.Mathai Scott.McNulty Mitch.Peck Robert.OConnell Andrew.Bender

---

Context & Problem Statement

Several entity types in the PLM platform expose `businessPartners` and `droppedPartnerIds` fields. These fields are resolved via a two-hop call through the **Access Control (ACL) Service** and the **VMM (Vendor Master Management) Service**:

1. Query the ACL Service for permissions on the resource — filter for grantees of partner types (`bps`, `MerchVendor`, `FabSupplier`, `TrimSupplie`, `dps`)
2. For each partner grantee ID, hydrate the full `BusinessPartner` object from VMM

`droppedPartnerIds` is a simpler variant — it reads the ACL service for `droppedIds` on matching permission entries and returns raw IDs without VMM hydration. As part of this work, the field will be promoted to a full entity type in the federated schema: `droppedPartnerIds: [ID] droppedPartners: [BusinessPartner]`, consistent with how `businessPartners` is already modeled.

This pattern is implemented in the legacy monolith as `loadBusinessPartners` and `loadDroppedBusinessPartners` in `commonResolvers/Teams.js`, and is shared across multiple entity types. As the platform migrates to a federated supergraph, these fields are absent from federated subgraph schemas and a decision is needed on where this resolution logic lives.

Affected Fields & Domains

Entity Type	Field	Pattern	Category
Wash, Color, Artwork, Trim, Fabric, Hub, Palette, Combination	`businessPartners`	ACL VMM hydration	Material
Wash, Color, Artwork, Trim, Fabric, Hub, Palette, Combination	`droppedPartnerIds` `dropped Partners`	ACL dropped IDs full `BusinessPartner` entity (schema improvement)	Material
`BOM` / `BOM_Unified`	`businessPartners`	ACL VMM hydration (via `bomUtils`)	Non-material
`SearchCombination`, `SearchPalette`, `SearchMaterial`, `SearchWatchlist`	`businessPartners` / `dropped PartnerIds`	ACL VMM hydration	Search projections

Note: `Product`, `Workspace`, `Impression`, `Measurement`, and `ProductDetails` also expose `businessPartners` but resolve them differently — partner IDs are embedded in the service response payload and hydrated directly from VMM without an ACL lookup. Those types are out of scope for this ADR.

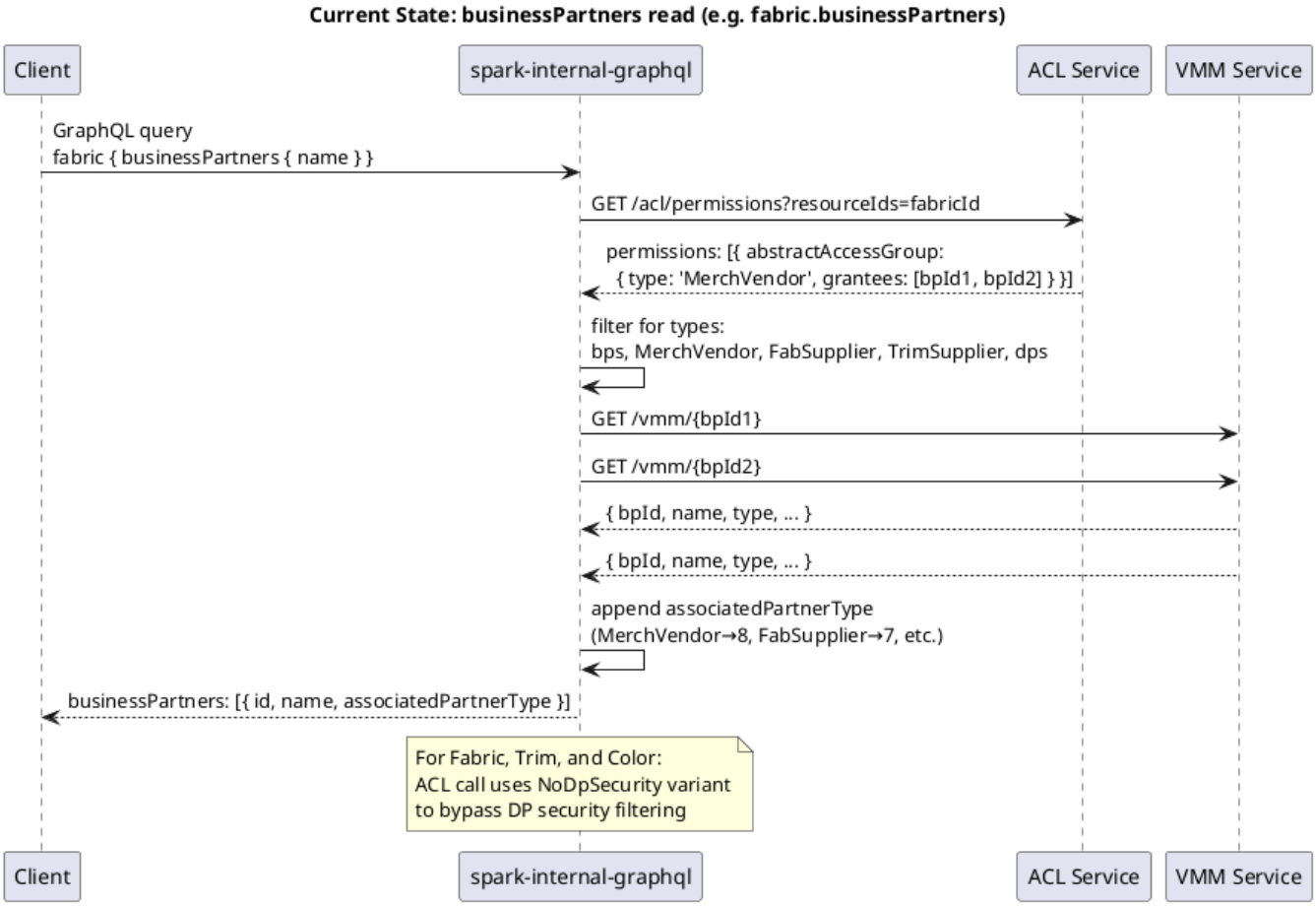
Existing Federation State

`BusinessPartner` is already a live federated entity in the supergraph:

Type	Owner Service	Federation Key	Notes
`BusinessPartner`	`plm-adapter/business-partner-service`	`@key(fields: "id")`	`__resolveReference` wired in `BusinessPartnerWiring.kt`

The question is not how to define `BusinessPartner` — it already exists — but rather **which subgraph is responsible for resolving the ACL lookup that produces the list of partner IDs** associated with a given resource.

Current State — Read Flow

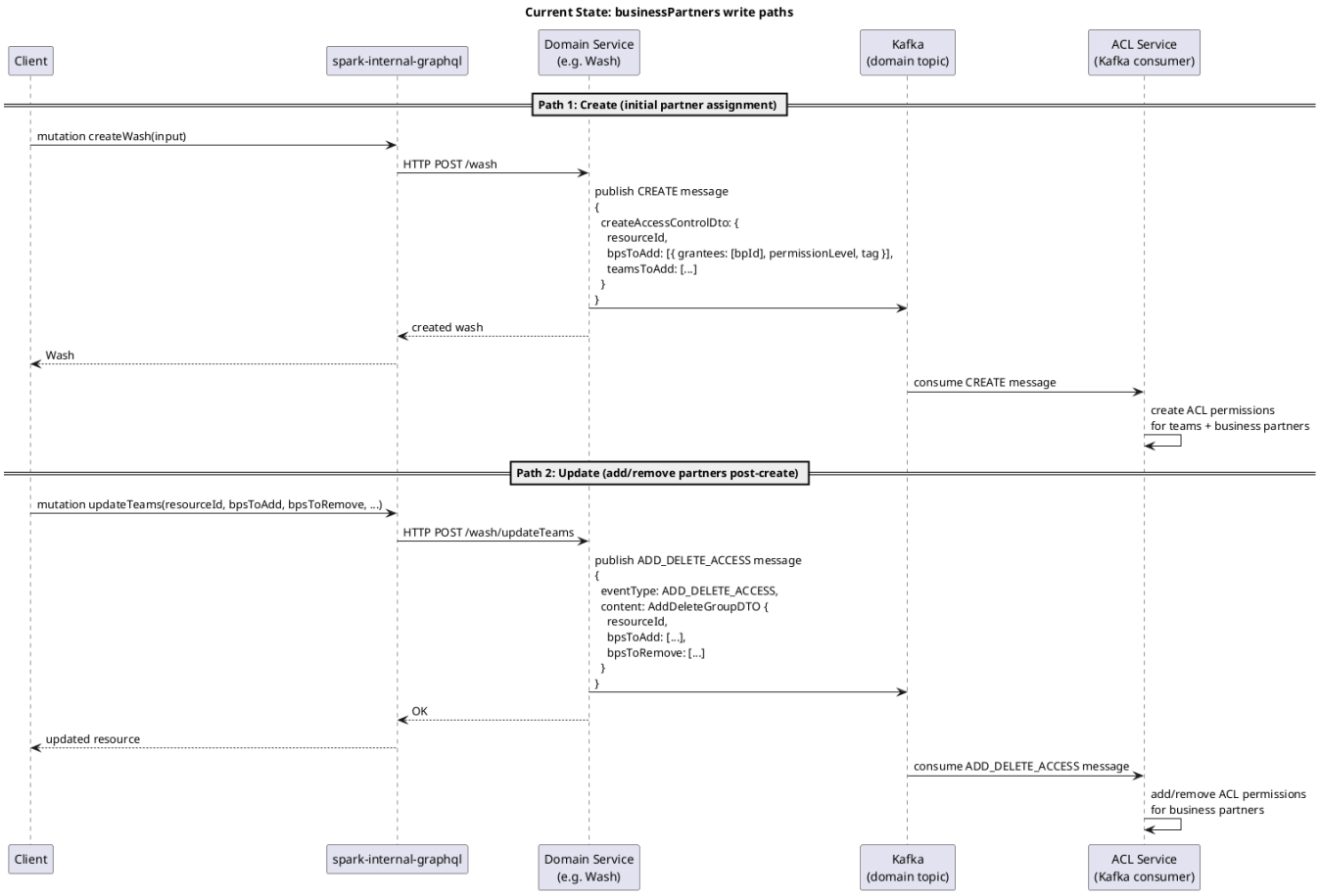


Current State — Write Flow

There are two distinct write paths for business partner ACL permissions:

Path 1 — Create (initial partner assignment): Business partners are embedded in the Kafka create message via `createAccessControlDto`. The ACL Service consumes this and creates permissions directly — the monolith is not involved.

Path 2 — Update (add/remove partners post-create): The `updateTeams` mutation is used for all post-create partner changes. The domain service publishes an `ADD_DELETE_ACCESS` Kafka message; the ACL Service consumes it.

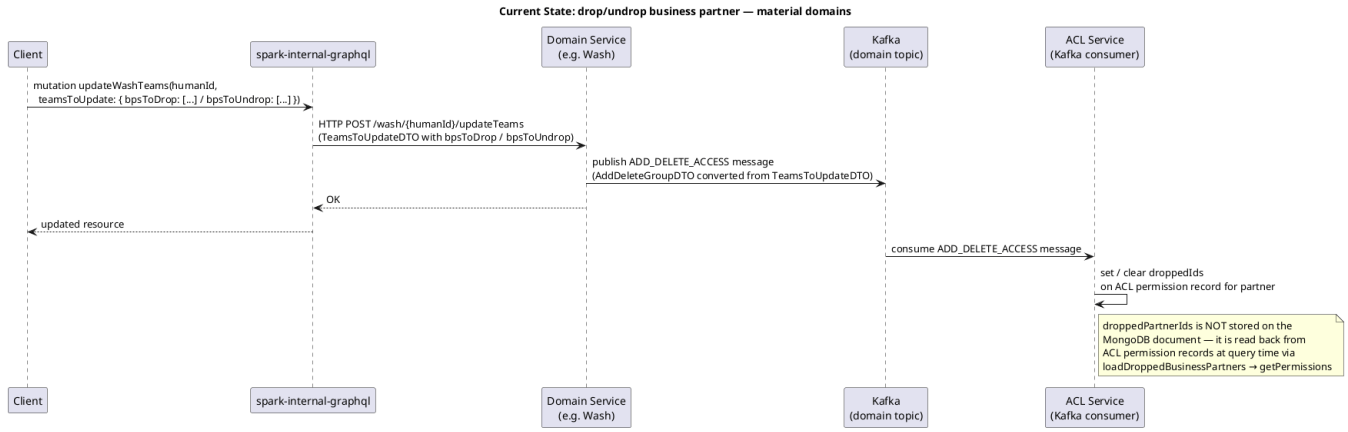


Current State — Drop / Undrop Flow

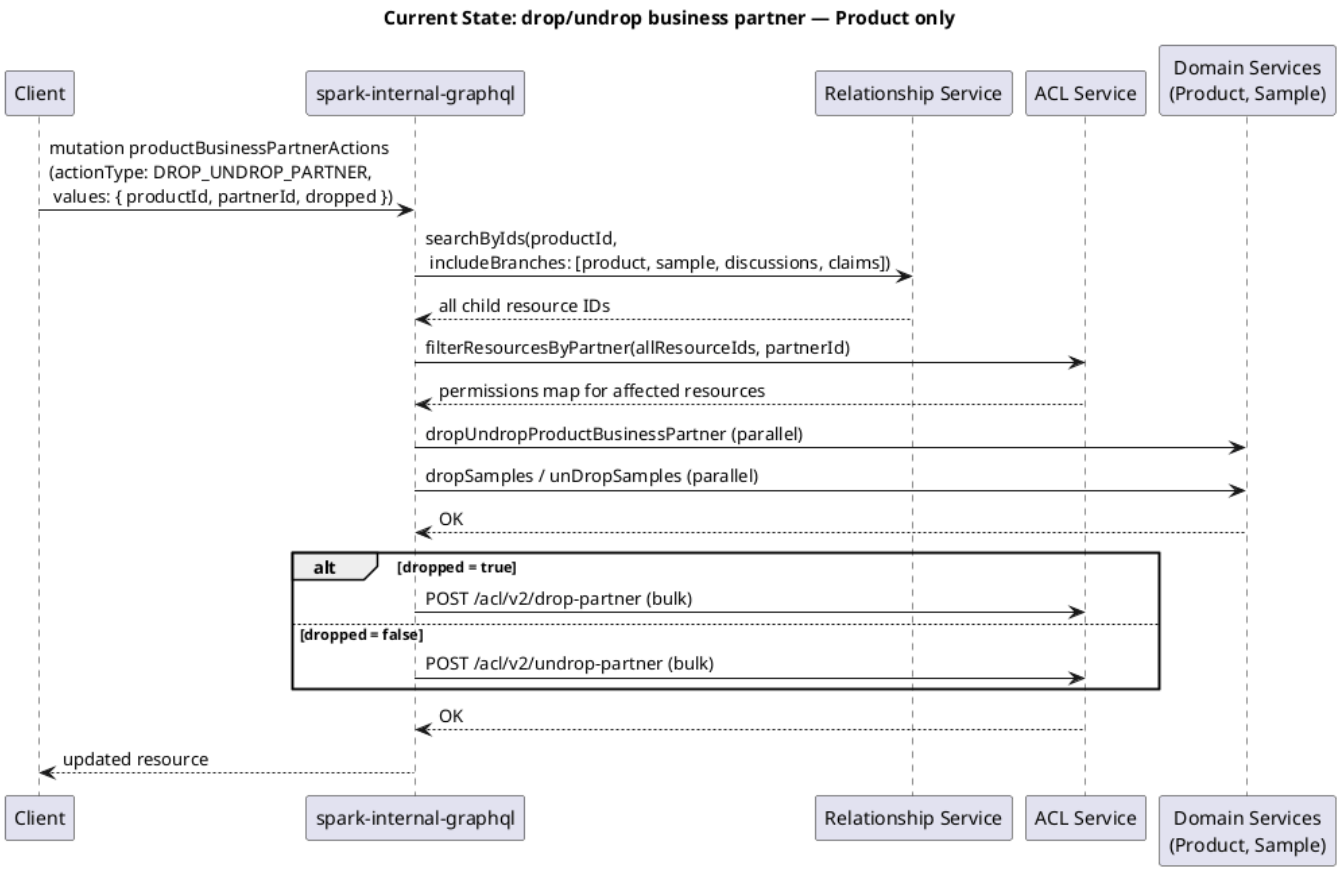
Drop/undrop behaves **differently** for material domains vs. Product. Material domains do not store `droppedPartnerIds` on the MongoDB document — it is derived entirely from ACL permission records (`droppedIds` on the ACL group). Drop/undrop flows through the same `updateTeams` endpoint as add /remove, via `bpsToDrop` / `bpsToUndrop` fields on `TeamsToUpdateDTO`.

Product has a much more complex drop flow involving Relationship Service traversal across child resources (samples, discussions, claims) and direct HTTP calls to the ACL Service — this is **not applicable to material domains**.

Material Domains (Wash, Color, Fabric, Trim, Artwork, Palette, Combination, Hub)



Product (for reference — not in scope for this ADR)



The Federation Gap

In the federated supergraph, each domain is a separate DGS subgraph. The shared `loadBusinessPartners` utility does not exist in the federated layer. A decision is needed on where the ACL lookup logic lives when a federated subgraph needs to resolve `businessPartners` or `droppedPartners`.

Related: `getUserAccessUnencoded` also needs a federated home

The UI's edit authorization pattern (`hasWrite` check) is powered by `getUserAccessUnencoded` — a query that returns the current user's permission level on a resource. In the legacy monolith it is co-located in the same GraphQL query as the material fetch (e.g. `getWash + getUserAccessUnencoded` in a single round-trip), and is used universally across all domains to gate edit controls, forms, and mutations.

As the platform migrates to the federated supergraph, `getUserAccessUnencoded` will also need a federated home — it cannot remain monolith-only once the UI cuts over to `plmClient`. This is a related but distinct concern from `businessPartners` resolution.

This has direct bearing on the option selection: if `access-control-service` becomes a federated subgraph (Option E), it is the natural home for ge `tUserAccessUnencoded` as well — making it the single subgraph for all ACL-derived fields. Options A–D do not address `getUserAccessUnencoded` and would leave it unresolved, requiring a separate decision. This should be weighed when evaluating the options below.

Broader ACL Federation Context

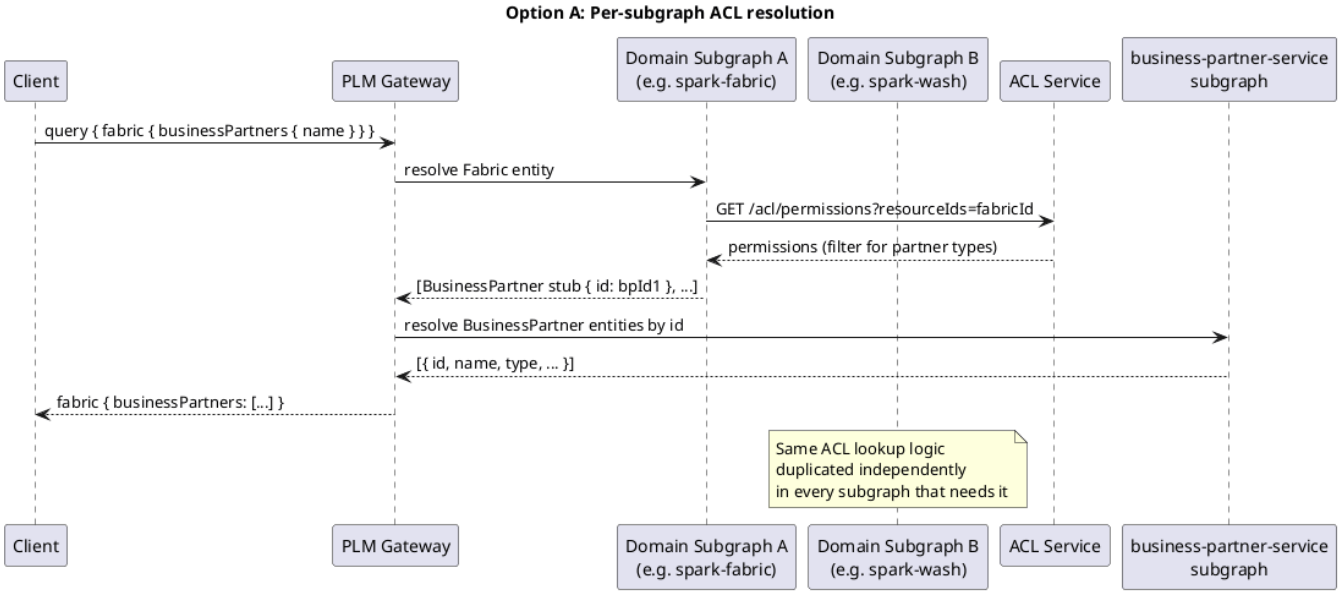
The ACL service is one of the most heavily used cross-cutting services in the platform. The following use cases all currently live in the `spark-internal-graphql` monolith and will each need a federated home as the migration progresses. This is not exhaustive of the full migration effort but is relevant context for evaluating the options below.

Use Case	ACL Method(s)	Needs Federated Home	Notes
UI `hasWrite` / permission level check	`getUserAccessUnencoded`, `getUserAccess`, `getUserAccessByProxy`	Yes — blocks UI cutover to `plmClient`	Co-located with entity fetches in every domain query today
Partner association reads	`getPermissionsForResource`, `getPermissionsForResourceNoDpSecurity`, `getPermissions`	Yes — this ADR	Powers `businessPartners` / `droppedPartners`
ACL group management	`createAccessGroupInternal`, `createAccessGroupAD`, `createAccessGroupID`, `createAccessGroupBP`, `addAccessGroupIds`, `deleteAccessGroupIds`, `updateAccessControlGroups`, `getAccessControlGroups`	Yes — admin UI depends on these	Exposed as public mutations /queries via `SPARK_AccessControl`
Resource permission grants/revokes	`createAccessControl`, `createBulkAccessControl`, `addAccessControlPermission`, `deleteAccessControlPermission`, `updateAccessControl`, `bulkUpdateAttachmentPermissions`	Yes — write path for teams /BPs	Used by Attachment, Discussion write flows
Resource locking	`lockResource`	Yes	Exposed as a mutation via `SPARK_AccessControl`
Drop/undrop partner	`dropPartnerFromResources`, `unDropPartnerFromResources`	Yes — this ADR	Used by Product, Workspace, material domains
Capability JWT generation	`getUserAccessByPost`	No — already solved	Moved to the domain service layer; no longer a concern for the GraphQL layer

The breadth of this inventory reinforces the case for **Option E** — making `access-control-service` a federated subgraph is not just a solution for `businessPartners` resolution, it is a prerequisite for the broader platform migration. A decision to pursue any other option for this ADR does not eliminate the need to eventually federate the ACL service for the remaining use cases above.

Option A — Each domain subgraph resolves businessPartners independently

Each subgraph that needs to resolve `businessPartners` adds its own ACL Service client and implements the lookup and VMM hydration logic directly, returning `BusinessPartner` stubs by ID for the federation router to resolve.



Pros:

- No new subgraph or shared infrastructure required
- Authz security concerns are handled automatically, only permissions granted to the logged in user are used for determining the BP list
- Each domain team has full ownership and control of their resolver
- Straightforward to implement per domain in isolation

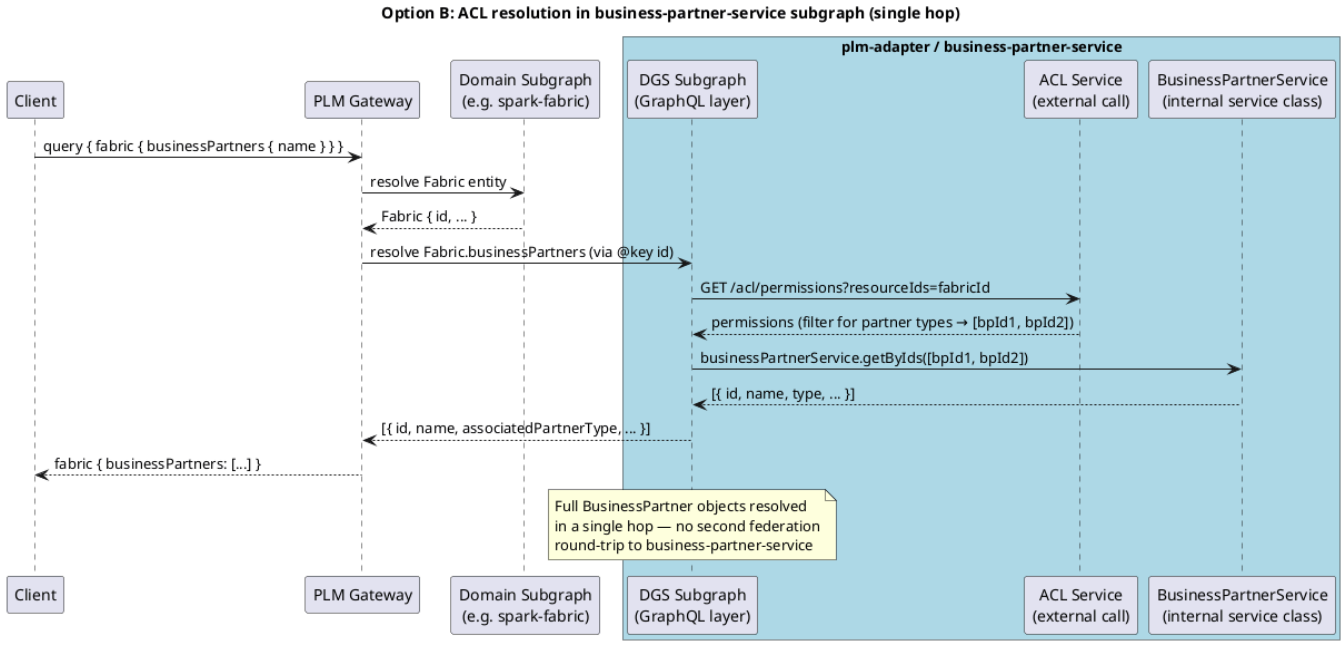
Cons:

- ACL lookup logic is spread across many subgraphs; while a shared library within the `plm-materials` monorepo could centralize utilities, each subgraph still requires its own wiring and configuration
- Changes to the ACL Service API, partner type filtering rules, or the `NoDpSecurity` branching logic require updates across subgraphs; the concern grows for domains outside `plm-materials` (e.g. BOM in a separate repo) — there is no shared enforcement point

---

Option B — ACL resolution added to `business-partner-service` subgraph

The existing `business-partner-service` subgraph (in `plm-adapter`) is extended to also resolve `businessPartners` and `droppedPartners` on entity types via `@key`. Since this service already owns the `BusinessPartner` entity and already integrates with ACL and VMM, it can perform the ACL lookup and resolve the full `BusinessPartner` objects in a single hop — no second federation round-trip needed.



Pros:

- No new service or subgraph required — extends an existing one
- `business-partner-service` already owns `BusinessPartner` and can resolve the full entity in a single hop without a second federation round-trip, once it performs the ACL lookup to obtain the partner IDs
- A single place to update if the ACL Service API or partner type filtering changes
- Acts as a single enforcement point for all entity types, including non-material domains (e.g. BOM) that use the same ACL pattern

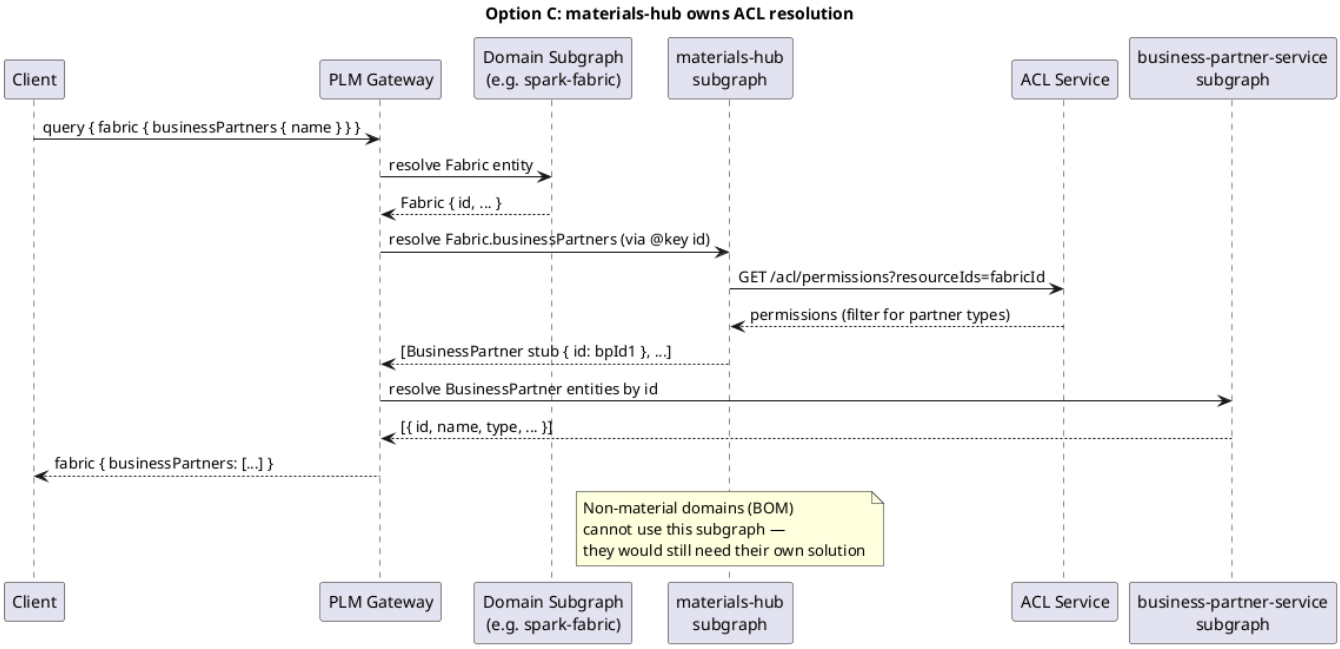
Cons:

- Extends the scope of `business-partner-service` beyond owning the `BusinessPartner` entity — it now also owns the ACL association lookup for other entity types
- Must be kept in sync with `@key` definitions across all participating domain subgraphs as new entity types are onboarded

---

Option C — Existing `materials-hub` subgraph owns cross-cutting ACL resolution

The existing `materials-hub` subgraph is extended to resolve `businessPartners` and `droppedPartnerIds` for all material entity types.



Pros:

- No new subgraph required
- Reuses existing infrastructure and deployment pipeline

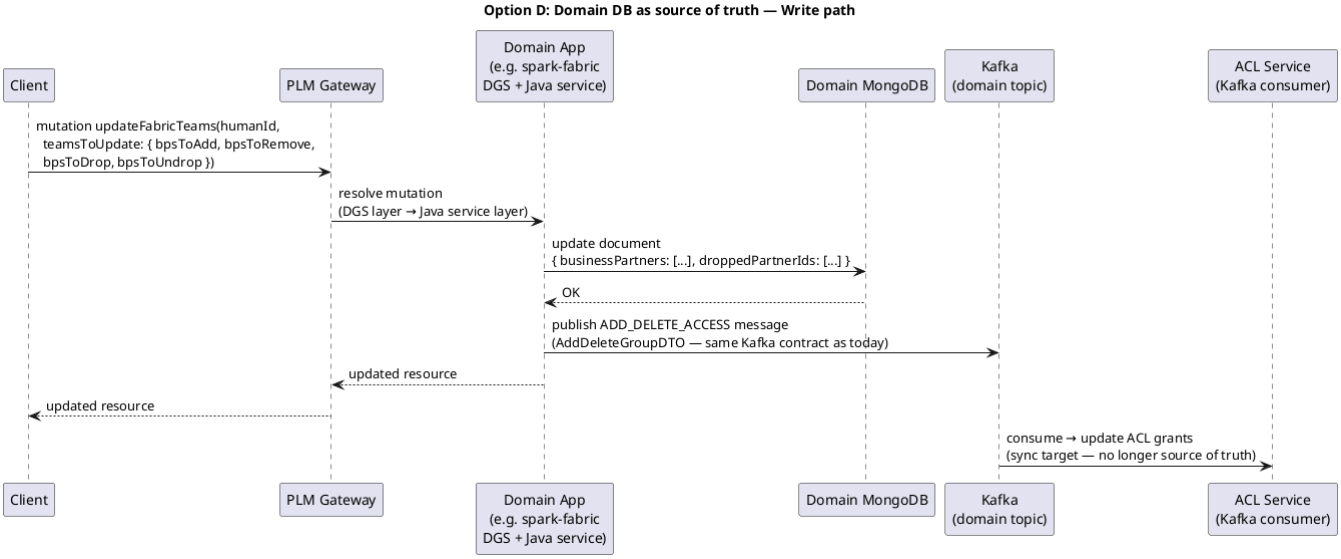
Cons:

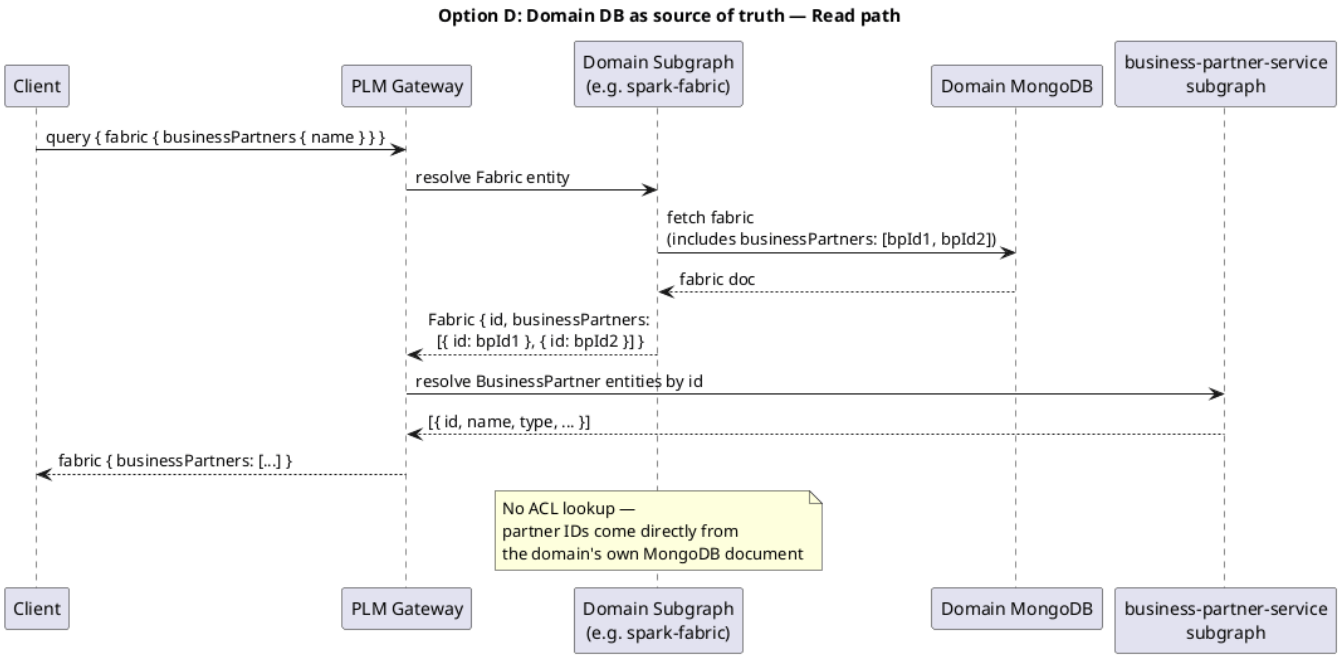
- Conflates the Hub/BaseMaterial domain with a cross-cutting platform concern
- Non-material domains (BOM) cannot use this subgraph — they would still need their own solution
- Changes to ACL resolution logic are coupled to the Hub domain release cycle

---

Option D — Domain services become the source of truth; ACL becomes a sync target

Each domain service stores partner associations directly on its own entity document (e.g. `fabric.businessPartners: [bpId]`). The ACL Service is retained as the authorization enforcement layer but is no longer the source of truth for the association list. Domain services write partner associations to their own data store and keep the ACL Service in sync via the existing Kafka event stream.





Pros:

- Eliminates the ACL lookup at query time — partner associations are co-located with the entity
- Write path is additive — domain service already publishes Kafka messages; the only change is also persisting partner IDs to the domain document
- ACL Service is preserved as the authorization enforcement layer — no disruption to access control behavior
- Read cutover can be done incrementally per domain
- Aligns ownership: the domain service that owns the entity also owns its partner associations

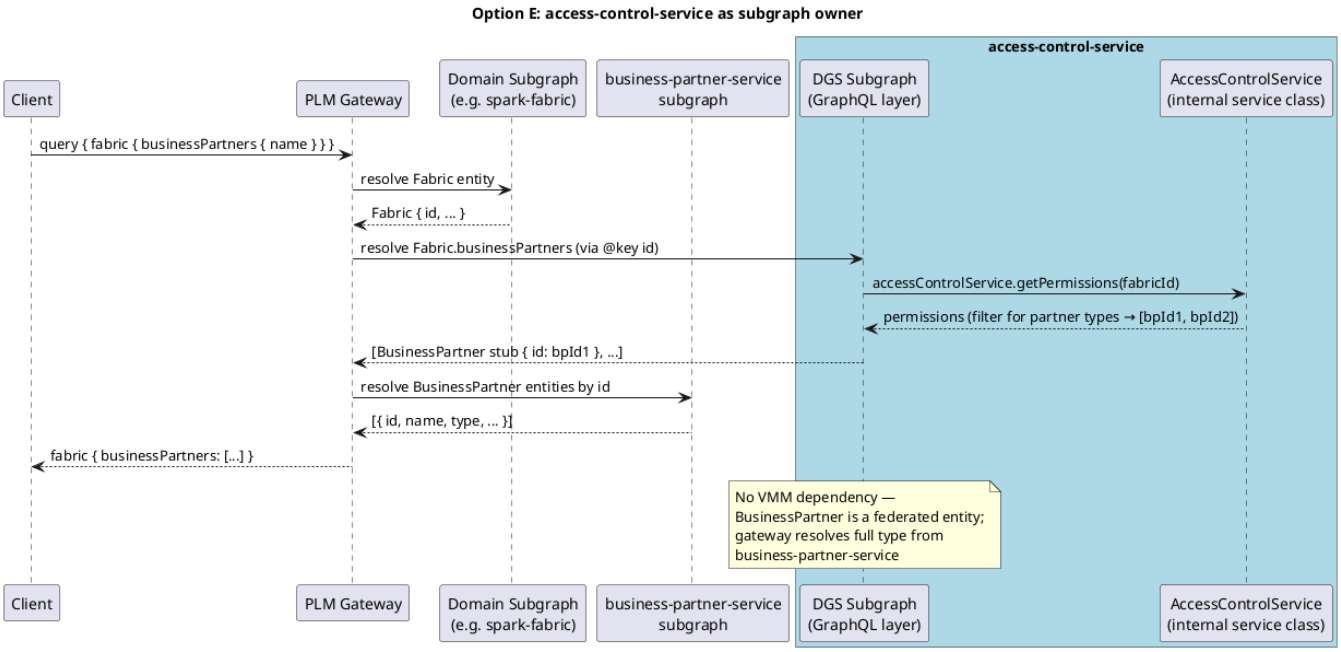
Cons:

- We still involve ACL for flows that need security. For UI flows that involve business partners, we have to filter business partners utilizing ACL. E. g. Hansae is viewing a wash asset, wash UI queries for wash.businessPartners, we don't want other businessPartners showing up in the GQL response in the network traffic.
- Requires a one-time data migration to backfill existing partner associations into each domain's MongoDB document
- Increases the scope of each domain migration story
- ACL Service must remain in sync — any write path that bypasses the domain service (e.g. direct ACL mutations) could cause drift between the domain DB and ACL state
- The `NoDpSecurity` branching logic (Fabric, Trim, Color) must be preserved in the read path or re-evaluated with the ACL/security team
- Does not address BOM or other non-material domains that use the same ACL pattern — they would still need a resolution strategy
- Source-of-truth shift requires broader team alignment, particularly with the ACL/security team

---

Option E — `access-control-service` adds a GraphQL subgraph and owns `businessPartners` / `droppedPartners` resolution

The `access-control-service` (Spark-owned) adds a DGS subgraph layer and uses Apollo Federation `@key` to extend entity types with `businessPartners` and `droppedPartners` fields. Since the ACL service is the actual source of truth for partner associations, it is the most semantically correct owner of this resolution. It returns `BusinessPartner` stubs by ID — no VMM dependency needed, as `BusinessPartner` is already a federated entity and the gateway resolves the full type from `business-partner-service`.



Pros:

- The ACL service is the actual source of truth for partner associations — this is the most semantically correct ownership model
- Authz security concerns are handled automatically, only permissions granted to the logged in user are used for determining the BP list
- No VMM dependency required in the ACL subgraph — `BusinessPartner` stubs are sufficient; the gateway resolves the full entity from `businesses-partner-service`
- No new service required — adds a GQL layer to an existing Spark-owned service
- A single enforcement point for all entity types across all repos (materials, BOM, Product, Workspace) — any entity type can be extended here
- `NoDpSecurity` branching logic is co-located with the service that owns it

Cons:

- Extends the ACL service's responsibilities beyond authorization into GraphQL field resolution — may blur the service's boundary
- Requires adding a GraphQL transport layer (DGS subgraph) to the existing `access-control-service`
- As new entity types are onboarded to the federation from other repos (e.g. Product, BOM), `access-control-service` must be updated to declare the corresponding entity stubs — a low-friction change but a coordination touch point across team boundaries
- `NoDpSecurity` branching behavior (Fabric, Trim, Color) needs to be confirmed with the ACL/security team before implementation

---

Evaluation Summary

Criteria	Option A	Option B	Option C	Option D	Option E
Implementation effort	Low per domain, high total	Medium (one-time)	Medium	High (migration + per domain)	Medium (one-time)
Maintenance surface	High (many subgraphs)	Low (1 subgraph)	Low (1 subgraph)	Low (per domain, co-located)	Low (1 subgraph)
Mirrors monolith pattern	Partially	Yes	No	No — improves on it	Yes
Introduces new subgraph	No	No (extends existing)	No	No	Yes (new subgraph on existing service)
Domain ownership clarity	High	High	Low	High	High
Semantic correctness of ownership	Medium	Medium	Low	High	High
Risk of behavioral divergence	High	Low	Low	Low	Low
Cross-repo / cross-domain extensibility	Low	High	Low (materials only)	Low	High
Query-time performance	Same as today	Improved (single hop)	Same as today	Improved (no ACL hop)	Same as today

Requires data migration	No	No	No	Yes	No
ACL remains authoritative	Yes	Yes	Yes	Partial (sync target)	Yes

---

Decision Outcome

Chosen option: Option A — Each domain subgraph resolves businessPartners independently

Rationale:

- Aligns with the present day domain service :: acl interfaction where domain GQL resolvers resolve this data (businessPartners, droppedBusinessPartners)
- Domain service is fully responsible for providing a fully authorized domain entity to the platform. Delegating authz to ACL service remains a private implementation detail
- ACL service remains free of Domain logic and data types

Consequences

TBD — to be filled in after decision is made.

Follow-Up Work

- [ ] Confirm ACL Service auth pattern for calls from DGS subgraphs
- [ ] Clarify `NoDpSecurity` branching requirement for Fabric, Trim, and Color with the ACL/security team — determine if this must be preserved in federation
- [ ] Inventory all non-PLM-materials consumers of the ACL partner pattern before any source-of-truth shift (relevant to Option D)
- [ ] Update all 8 material domain backend stories to reference this ADR for `businessPartners` and `droppedPartnerIds`
- [ ] Update SDL across all affected subgraphs: `droppedPartnerIds: [ID] droppedPartners: [BusinessPartner]` — return stubs by ID, federation resolves full entity
- [ ] Align with BOM team on whether their `businessPartners` field follows the same resolution path in federation

---

References

- SPARK-30228 — ADR Jira story
- ADR-010 — Relationship Service association resolution (related)
- Materials on GQL Federation - Generic Subgraph
- `spark-internal-graphql/packages/data-source-spark/src/resolvers/commonResolvers/Teams.js`
- `spark-internal-graphql/packages/data-source-spark/src/services/AccessControl.js`
- `plm-adapter/apps/business-partner-service/src/main/resources/schema/businesspartner.graphqls`
- `plm-adapter/apps/business-partner-service/src/main/kotlin/com/target/plm/graphql/BusinessPartnerWiring.kt`